

Automated Software Testing Part 2  
Automated Reduction of  
Bug-Triggering SQL Queries

Edouard Mignan (21806807)  
Eduard Mihai (25933516)

June 11, 2026

# 1 Technical Description

In this section we will describe the methods implemented in our reducer and the limitations of our approach.

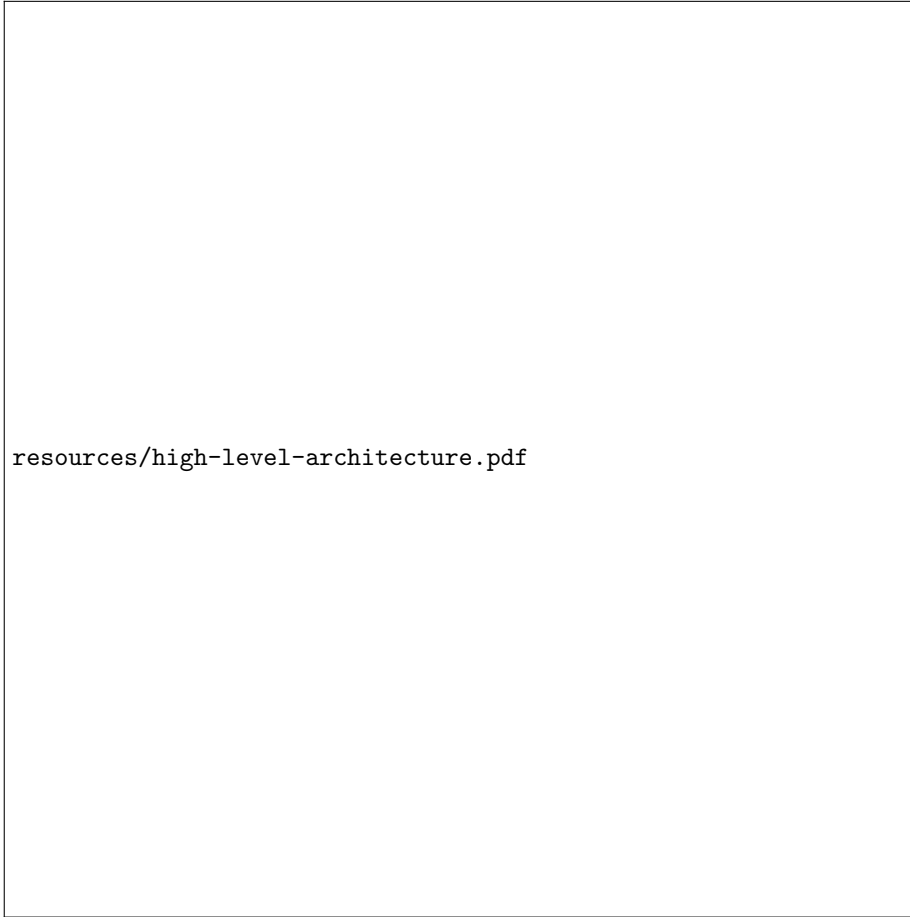
Our reducer is an implementation of Hierarchical Delta Debugging [1] (particularly, a variant called HDD\*). Contrary to Delta Debugging [2], which does not use any knowledge of the structure of an input to perform reduction, Hierarchical Delta Debugging (HDD) applies to tree-structured inputs and uses knowledge of that structure to perform better reductions with less generated test cases. Our reducer works by parsing a SQL query into a tree (Abstract or Concrete Syntax Tree) and applying Delta Debugging to each level of that tree, from the highest to the lowest, in order to prune irrelevant nodes, and finally reconstruct the reduced query.

We implemented HDD\* (HDD with multiple passes until no more reduction is possible) with some termination criteria to improve performance. In particular, our reducer will compare changes between the beginning and the end of a pass and stop prematurely if the improvement is lower than 1%.

We tested multiple tools to parse SQL for our reducer and after some benchmarks, settled on ANTLR4, as the implementation using it had the best performance and reduction quality on average. The implemented fuzzer relies on the mutation-based greybox fuzzing paradigm presented in the lectures. It starts from the provided seed files and uses a lightweight coverage feedback to guide the exploration of the input space, combined with differential testing as the test oracle. Its goal is to find logical bugs, crashes, and unexpected errors in a patched version of SQLite by comparing its output against a reference binary. On a high level, the system is composed of three main components, as illustrated in Fig 1.

1. **Mutation Engine:** Mutations are applied at the Abstract Syntax Tree (AST) level, which significantly increases the probability that mutated queries remain syntactically valid. We use the `sqlglot` library [?] to parse, serialize, and modify SQL queries, taking advantage of its native support for the SQLite dialect. However, syntactic correctness does not guarantee semantic validity (e.g., the query can still reference non-existent columns, use operators that are not suitable for the data types of the operands, violate unique constraints, and so on). Therefore, generating mutations that are both semantically valid and capable of exposing bugs was a primary challenge in this implementation.

To address this, we designed a collection of carefully handcrafted mutation rules. Given its wide spread, we chose the `SELECT` statement as the first contender, initially incorporating the standard replacements proposed in [?]. On top of that, we extended them with schema-aware modifications. More precisely, we dynamically computed the schema of the currently active block of statements by scanning the `CREATE TABLE` statements. By



resources/high-level-architecture.pdf

Figure 1: High-level components of the fuzzer.

having the existing columns, along with their data types, we could, for example, replace an identifier with another valid one or replace it with a literal of the same type. On this note, using boundary or extreme values for replacements turned out to uncover a few bugs.

The mutation process is based on small incremental modifications made to the initial seed files. To optimize efficiency, the engine analyzes each query selected from the corpus, identifying and bookkeeping all applicable mutation rules. A subset of these valid rules is then randomly selected to generate new query variants. If a query leads to no applicable mutations, it is pruned from the corpus.

2. **Test Oracle:** It is based on differential testing. There exists a binary that serves as the reference implementation. We feed the same SQL query

to both the reference binary and the target binary, and their outputs are compared. Any divergence signals a potential bug in one of them. The test oracle classifies each execution into one of the following six distinct outcomes: `SUCCESS`, `LOGICAL_BUG`, `CRASH`, `UNEXPECTED_ERROR`, `INVALID`, or `TIMEOUT`. To ensure robust crash detection, we rely on AddressSanitizer (ASAN) [?] and UndefinedBehaviorSanitizer (UBSAN), which are both compiled into the target binary. Furthermore, when both binaries execute without errors, since the result sets may be returned in non-deterministic row order, we sort the outputs lexicographically before comparing them. We address the limitations of this approach in Section ??.

3. **Coverage Tracker:** The target binary is compiled with GCC coverage instrumentation (`gcov`). We want to monitor the basic blocks covered by each query. However, instead of invoking the `gcov` library after every query execution, which is slow, we read the counters stored in the `.gcda` binary file directly. Before executing a query, we snapshot all counter values, and after execution, we read the updated values and compute deltas per counter. Moreover, coverage guidance is determined by the bucket schema proposed by AFL [?]. If a counter’s new bucket index exceeds its current maximum, the coverage tracker will record the new bucket index and add the corresponding query to the corpus. This approach captures both the discovery of new basic blocks and the increase in execution frequency of existing ones (similar to how loop coverage distinguishes between paths with different numbers of iterations).

After the fuzzer finishes, we use the instrumented binary to get statistics such as line, branch, and function coverage, which we’ll report in Section 2.

## 2 Evaluation

### 2.1 Bug-Finding Capabilities

Overall, the fuzzer was able to find 8 different bugs (7 logical mismatches and 1 unexpected error). We documented them in the `bug-reproducers` directory.

### 2.2 Query Generation Characteristics

After generating 10,000 queries, we get the following statistics concerning the frequency of appearance of SQL keywords in Table 1 and Table 2, and the coverage in Table 3. As we can see, the `SELECT` and `INSERT` keywords cover a very high number of queries (a bit over 90%) and also appear on average 2.45 and, respectively, 2.34 times per query.

We also obtain good results (97.09%) in terms of query validity, showing that our mutator is highly capable of generating valid queries that can then explore and cover SQLite code.

The frequency and validity statistics are reproducible by using the seed 42.

| SQL Keyword | Query coverage | SQL Keyword | Average frequency |
|-------------|----------------|-------------|-------------------|
| CREATE      | 10000          | FROM        | 3.34              |
| TABLE       | 10000          | CREATE      | 2.96              |
| FROM        | 9098           | TABLE       | 2.47              |
| SELECT      | 9098           | SELECT      | 2.45              |
| INSERT      | 9073           | INSERT      | 2.34              |
| INTO        | 9073           | INTO        | 2.34              |
| VALUES      | 9073           | VALUES      | 2.27              |
| WHERE       | 6062           | AS          | 1.35              |
| JOIN        | 5341           | WHERE       | 1.35              |
| ON          | 5037           | BY          | 1.15              |
| BY          | 4887           | ON          | 0.92              |
| ORDER       | 4552           | JOIN        | 0.91              |
| AS          | 4540           | ORDER       | 0.78              |
| CROSS       | 3163           | NOT         | 0.58              |
| INDEX       | 2973           | INDEX       | 0.51              |
| PRIMARY     | 2717           | CROSS       | 0.47              |
| OR          | 2604           | OR          | 0.44              |
| NOT         | 2538           | PRIMARY     | 0.44              |
| AND         | 2400           | AND         | 0.42              |
| VIEW        | 2121           | IS          | 0.41              |
| DISTINCT    | 2078           | IN          | 0.35              |
| NULL        | 1994           | UNION       | 0.29              |
| KEY         | 1922           | GROUP       | 0.28              |
| REFERENCES  | 1844           | VIEW        | 0.28              |
| IS          | 1771           | NULL        | 0.28              |
| UNION       | 1605           | DISTINCT    | 0.26              |
| IN          | 1565           | EXISTS      | 0.26              |
| EXISTS      | 1540           | IF          | 0.26              |
| IF          | 1540           | KEY         | 0.26              |
| DROP        | 1393           | ALL         | 0.25              |

Table 1: Top 30 most frequent SQL keywords by the number of queries in which they appear at least once.

Table 2: Top 30 most frequent SQL keywords by the average frequency of appearance inside a query.

|         | Number of queries | Percentage of queries |
|---------|-------------------|-----------------------|
| Valid   | 9709              | 97.09%                |
| Invalid | 291               | 2.91%                 |

Table 3: Query validity statistics.

## 2.3 Code Coverage

We obtain the following coverage report by running `lcov` after executing 10,000 queries, illustrated in Table 4.

The coverage statistics are reproducible by using the seed 42.

| Type      | Coverage (%) |
|-----------|--------------|
| Lines     | 37.5         |
| Functions | 42.3         |
| Branches  | 27.6         |

Table 4: Achieved coverage after 10,000 queries.

## 2.4 Performance

To evaluate the performance of our fuzzer, we report two statistics: total throughput (the number of queries that can be generated and executed in a minute) and generation throughput (the number of queries that can be generated in a minute).

Since our approach is mutation-based and not strictly generation-based, the creation of new queries is reliant on the execution of previous queries and the feedback provided by the coverage tracker. In order to calculate the generation throughput and obtain a metric that reflects how fast the query generation is on its own, we have opted to record the time spent in the `mutate` function of the mutator class and divide the number of generated queries by that time.

To generate and execute 10,000 queries, the fuzzer took 10 minutes and 47 seconds while running in a Docker container on an 8-core AMD Ryzen 7 7735HS with 16GB of RAM.

As we can see from the data in Table 5, the bottleneck for our fuzzer performance-wise seems to be the differential execution of queries rather than the generation of new queries by mutation, with the generation throughput being about 40 times higher.

|                        | Throughput (queries/min) |
|------------------------|--------------------------|
| Generation             | 40963                    |
| Generation + Execution | 926                      |

Table 5: SQLite fuzzer performance on 10,000 queries.

## References

- [1] Ghassan Misherghi and Zhendong Su. HDD: Hierarchical delta debugging. In *Proceedings of the 28th International Conference on Software Engineering*, ICSE '06, pages 142–151, 2006.
- [2] Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.